

FOCI projekt – fejlesztői átadási dokumentum 4

Állapot: 2026-04-03 **Fókusz:** event-alapú, mentett csapatleosztás; stabil pilot-közel működés; következő chat és következő fejlesztő számára pontos átadás.

Összinté megjegyzés: a feltöltött 202604030252_foci-backend.zip alapján a snapshot hiányosnak tűnik: a gyökérben migrations, PDF-ek és node_modules látszanak, de a `src/public` forrásfa nem. Emiatt a jelenlegi működő frontendállapot forrásigazságát a külön feltöltött `app.js`, a korábbi átadók és a manuális CMD-tesztek alapján kell kezelni.

1. Mit néztem át

- a 2026-03-29-es átadási PDF-et
- a 2026-03-31-es fejlesztői átadási PDF-et
- a 2026-04-01-es 3. átadási PDF-et
- a feltöltött 202604030252_foci-backend.zip tartalmát
- a külön feltöltött jelenlegi `app.js` állapotot és az ehhez kapcsolódó admin/user draw logikát
- a beszélgetés során manuálisan validált event preview / save / get backend láncot

2. Rövid vezetői összefoglalás

A projekt már túl van a sima baráti foci CRUD-on. A backend szervezési mag erős, a legfontosabb üzleti logikák elkészültek: auth, team domain, invite-only belépés, captain transfer, event state machine, waiting list + auto-promotion, recurring events, team skill backend és most már event-alapú, mentett csapatleosztás is.

A legnagyobb előrelépés a 3. átadó óta az, hogy a csapatleosztás már nem csak kliensoldali preview: eseményhez menthető, visszatölthető, és a felhasználók számára is megjeleníthető. Ez lényegében átfordította a funkciót demóból termékszerű állapotba.

A legnagyobb fennmaradó kockázat nem a domainlogika, hanem a forráskezelési és szállítási fegyelem. A public réteg mikrolépéses javításokkal már kezelhető, de a snapshotok közötti inkonzisztencia továbbra is valós kockázat.

3. Mi változott a 3. átadás óta

- A team skill backendre ráépült az admin skill UI a team member kártyákban: egyéni skill ON/OFF, védő/támadó/kapus csúszkák, mentési flow.
- Elkészült a skill-alapú csapatsorsolás preview, majd ennek event-alapú változata is.
- Bevezetésre került a kapusjelölés szabálya: csapatonként a legmagasabb nem null `goalkeeper_score` kapja a (K) jelölést.
- Elkészült az event alapú csapatleosztás backend lánc: preview → save → get.
- A frontendben megjelent az admin oldali csapatleosztás blokk, a fókusz esemény logika, valamint a mentett draw külön állapotként történő kezelése.
- A user oldalon megjelent a mentett csapatleosztás megjelenítése, a skill pontok nélkül, két oszlopos csapatnézetrel.
- A soron következő esemény fókuszlogikája megjelent admin oldalon, és részben user oldalon is, de ez a szál további finomítást igényelhet.

4. Jelenlegi tényleges állapotkép

Backend biztosan kész / validált

- JWT auth; register/login/me
- team create/read
- team member add / role update / remove
- captain transfer
- invite create/list/revoke + accept/decline
- event create/read/update/status
- register / cancel / waiting_list / auto-promotion
- recurring events backend
- request validation
- auth rate limit
- startup config check
- team skill settings + member skill mentés
- event-alapú team draw preview
- event-alapú team draw save
- mentett event draw lekérése

Frontend biztosan látható / működő blokkok

- auth view tiszta
- admin recurring create UI látszik
- admin invite lista csoportosított
- team member kártyák skill csúszkákkal bővítve
- admin summaryban fókusz esemény blokk
- admin oldalon csapatleosztás blokk (preview / mentett draw logika)
- user eseményrészlet nézet
- user oldalon mentett csapatleosztás skill pontok nélkül

Ami stratégiailag elkészült, de még nincs lezárva

- admin és user automatikus eseményfókusz logika finomítása
- csapatleosztás státuszgép (preview/saved/published/stale) még nincs implementálva
- lemondás utáni stale kezelés és admin újragenerálási döntési pont még TODO

5. Kiemelt üzleti és technikai tanulságok

- A csapatleosztásnak nem a teljes team aktív tagjaiból, hanem az adott esemény tényleges going résztvevőiből kell készülnie.
- A csapatleosztás csak akkor termékértékű, ha menthető és visszatölthető; a sima preview önmagában csak demo.

- A felhasználói nézetben a skill pontok és balansz metaadatok feszültséget generálnak, ezért a user oldalon ezek elrejtése helyes döntés.
- A public réteget teljes fájl overwrite-tal nem szabad módosítani. A baseline-védelem továbbra is kötelező szabály.
- A szervezői fókusz nem az újonnan létrehozott esemény, hanem a soron következő releváns esemény 24–72 órás horizonton.

6. Frissített értékelés (miről mire változott)

Szempont	Előző	Most	Változá	Indok
Backend core	91	94	+3	Az event-alapú preview/save/get lánc és a draw perzisztencia erősítette a magot.
Üzleti workflow-k	87	91	+4	A team draw már eseményhez kötött, menthető és visszaolvasható.
Frontend / UX	66	76	+10	Admin skill UI, preview blokkok, user oldali mentett draw és fókuszlogika kész.
Production readiness	52	56	+4	A működés stabilabb, de staging/backup/monitoring továbbra sincs lezárva.
Pilot readiness	84	90	+6	Az eseményhez mentett draw miatt a pilot sokkal hihetőbb és kezelhetőbb.
Change safety / szál- lítási fegyelem	43	58	+15	A mikropatch-fegyelem javult, de a snapshot-inkonzisztencia még valós kockázat.
Összesített lapot	ál- 79	84	+5	A projekt ma erős pilot-mag; a draw már termék-szerűbb, nem csak demo.

7. Részletes TODO lista (prioritási sorrendben)

1. **Event-alapú csapatleosztás preview/save/get teljes lezárása a futó public és a tényleges kódfa között.** Miért: a logika már működik, de a forrásnapshotsok közötti inkonzisztenciát meg kell szüntetni; a következő fejlesztőnek egységes, verifikált állapot kell.
2. **Admin oldalon a mentett draw és a preview állapot teljes szétválasztása.** Miért: fogalmilag preview $\neq$ saved. Ezt UI-ban, state-ben és szövegezésben is végig kell vinni.
3. **User oldalon automatikus fókusz a soron következő eseményre.** Miért: a cél, hogy a user gombnyomás nélkül lássa a számára releváns mentett csapatleosztást.
4. **Csapatleosztás státuszgép bevezetése: preview / saved / published / stale.** Miért: kihirdetés után résztvevőváltozás esetén ne írjuk felül automatikusan a csapatokat; legyen elavult állapot és admin döntési pont.
5. **Stale jelölés triggerelése résztvevőváltozáskor.** Miért: ha valaki lemond és a várólistáról valaki bekerül, a meglévő draw elavulhat. Ezt explicit jelezni kell.
6. **Admin figyelmeztetés stale draw esetén.** Miért: a szervező lássa, hogy a kihirdetett csapat már nem pontos, és újragenerálásra lehet szükség.
7. **Automatikus újragenerálás tiltása published draw esetén.** Miért: a rendszer ne borítsa fel csendben a már kommunikált csapatokat.
8. **Admin skill UI finomítása és a globális skill modul ON/OFF csapatszintű kapcsoló bevezetése.** Miért: ez teszi a rendszert valóban használhatóvá skill nélkül is, például röplabda vagy kosár jellegű baráti eseményekre.
9. **Skill OFF fallback csapatgenerálás definiálása.** Miért: random vagy egyszerű balansz mód kell arra az esetre, ha a csapat nem használ skillt.

10. **Role-aware team draw későbbi tervezése.** Miért: a jelenlegi algoritmus overall balance-t csinál, de nem garantálja a támadók/védők tudatos szétosztását.
11. **Szezonális részvételi számlálás.** Miért: a rendszer számolja, ki hány alkalommal jött egy szezonon belül.
12. **Rangrendszer 10+ részvétel után.** Miért: későbbi prioritási és lojalitási logika alapja.
13. **Prioritás logika szűk férőhelyes eseményekhez.** Miért: régi, aktív tagok előnybe részesítése kis pályás, telített eseményeknél.
14. **Értesítési rendszer alap.** Miért: 5 nappal előtte, 3 nappal előtte, 1 órával előtte, várólista-promóció, státuszváltozás, draw publikálás.
15. **Naptárba mentés / ICS export.** Miért: ha a user going státuszú, legyen egyszerű naptárba mentés lehetőség.
16. **Event életciklus és no-show logika részletesítése.** Miért: külön kell választani a jelentkezést, tényleges megjelenést és lezárt esemény utóéletét.
17. **Mobilbarát vizuális finomhangolás.** Miért: a kétszlopos csapatnézetnek mobilon is egyértelműnek és olvashatónak kell maradnia.
18. **Teljes regresszióteszt-kör bővítése az új draw funkciókra.** Miért: jelenleg a legnagyobb technikai kockázat a frontend-state és a részleges snapshotok eltérése.
19. **Staging / deploy / monitoring / backup stratégia.** Miért: production readiness itt marad le leginkább.
20. **Multi-sport alapozás.** Miért: a termék ne zárja magát fociba; az event és a skill logika később legyen sportfüggetlenné tehető.
21. **Tornaszervező modul előkészítése.** Miért: a baráti modul hosszú távon 8/16/32 csapatos tornairányba bővíthető; ehhez a mostani event/részrtvevő logikát tisztán kell tartani.

8. Következő fejlesztőnek szóló munkafegyelem

- Egy patch = egy cél.
- A public réteghez csak izolált blokk-szinten szabad nyúlni.
- Teljes public fájl felülírás tilos.
- Minden patch előtt és után működési ellenőrzés kell.
- Ha egy javítás 1 dolgot megold, de 1 működő dolgot elront, azonnali rollback.
- A csapatleosztásnál a domain-igazság az event, nem a team.
- A user oldalon ne jelenjen meg skill pont, diff vagy tolerance metaadat.

9. Következő chatbe másolható prompt

FOCI projektet folytatjuk. Ez a kvetkez kln chat a projekten bell.

A rendszer jelenlegi llapota rviden:

- Node.js + Express + PostgreSQL backend
- JWT auth stabil
- team create/read, team member add/update/remove ksz
- captain transfer ksz
- invite create/list/revoke + accept/decline ksz
- event create/read/update/status ksz
- register/cancel/waiting_list/auto-promotion ksz
- recurring events backend + admin create UI ksz
- request validation + auth rate limit + startup config check ksz
- team skills backend ksz
- admin skill UI pilot szinten bent van
- event-alap team draw preview/save/get backend elkészlt
- admin oldalon van fkusz esemny logika s mentett/preview draw blokk
- user oldalon van mentett csapatleosztts megjelents skill pontok nlkl

Nagyon fontos munkaszably:

- egy patch = egy cl
- csak az adott blokkhoz nylj
- teljes public fjl fellrs tilos
- regresszi esetn azonnali rollback
- a csapatleosztts domainje esemnyhez kttt, nem teamhez ltalban

A jelenlegi legfontosabb nyitott pontok:

1. draw sttusgpg: preview / saved / published / stale
2. stale jells rsztvevvltozs utn
3. user oldali automatikus fkusz finomtsa a kvetkez relevns esemnyre
4. globlis skill modul ON/OFF csapatszinten
5. rtests rendszer s naptrba ments ksbbre
6. szezonlis rszvtelszmls s rangrendszer
7. role-aware team draw ksbbi fejlesztts
8. mobilbart UI finomts

Fontos zleti dntsek:

- user oldalon nem jelenhetnek meg a skill pontok vagy balansz metaadatok
- a kihirdetett csapat ne rja fell automatikusan magt rsztvevvltozskor
- ilyen helyzetben stale llapot + admin dnts kell
- a szervez fkusza mindig a soron kvetkez relevns esemny, nem a tvoli vagy frissen ltrehozott esemny

A kvetkez fejleszttsi lpst gy vlaszd meg, hogy ne bontsd meg a jelenlegi public baseline-t.
Innen folytassuk, ne ptsnk jra ksz auth / invite / waiting list / recurring logikt.

10. Záró ítélet

A projekt jelenlegi állapota már nem sima backend-MVP. A szervezési mag erős, a pilot használat valószínű, és a csapateleostás immár termékszerűen menthető és visszaolvasható.

A fő feladat innentől nem az alaplogika újraírása, hanem az üzemi fegyelem, a stale draw kezelés, a kommunikációs réteg és a használhatóság lezárása.

Összesített értékelés: 84/100. Erős pilot-alap, de még nem production termék.